

Stream Engines: Azure Stream Analytics

Repository Structure

The repository is organized as follows:

```
outputs/  
queries/  
screenshots/  
src/  
.gitignore  
README.md  
report.md  
requirements.txt
```

- `outputs/`: contains sample JSON output files downloaded from Azure Blob Storage after running the Stream Analytics jobs.
- `queries/`: contains the SQL queries used by the five Azure Stream Analytics jobs.
- `screenshots/`: contains the screenshots used to document the Azure setup, generator execution, Stream Analytics jobs, and Blob Storage outputs.
- `src/`: contains the Python event generator script.
- `.gitignore`: excludes local files that should not be committed, such as `.env` and the virtual environment.
- `report.md`: contains the detailed project documentation and implementation report.
- `requirements.txt`: lists the Python dependencies required to run the event generator.

Introduction

This project implements a real-time stream processing pipeline using Azure Event Hubs, Azure Stream Analytics, and Azure Blob Storage. The goal is to simulate ATM transaction events, process them in real time, and store the aggregated results as JSON files.

The overall architecture of the project is:

Python ATM Generator → Azure Event Hub → Azure Stream Analytics Jobs → Azure Blob Storage

The Python event generator simulates ATM transactions and continuously sends them to Azure Event Hub. Each generated event represents a single ATM transaction and includes fields such as the event timestamp, ATM area, ATM identifier, account identifier, transaction type, transaction amount, and currency.

Azure Event Hub acts as the ingestion layer of the pipeline. It receives the continuous stream of ATM transaction events and makes them available to downstream services. It does not perform any analysis itself; its role is to collect and hold the incoming events so they can be processed in real time.

Azure Stream Analytics is used as the stream processing engine. It reads the events from the Event Hub and applies SQL-like queries over time-based windows. Each Stream Analytics job follows the same basic structure:

Input → Query → Output

In this project, the input is the Event Hub, the query performs a specific aggregation or detection task, and the output is written to Azure Blob Storage.

Azure Blob Storage is used as the final storage layer. Each task writes its results to a separate Blob container in JSON format. This makes it possible to verify that the Stream Analytics jobs produced valid output files for all required tasks.

The project includes five Stream Analytics tasks:

1. Transaction volume per area using a tumbling window.
2. High-value withdrawals using a hopping window.
3. Burst activity detection per account using a sliding window.
4. Average withdrawal amount per area using a tumbling window.
5. Transaction type distribution per area using a hopping window.

The implementation demonstrates how real-time events can be ingested, processed with window-based stream queries, and stored automatically as structured output files in Azure.

Azure Environment Setup

1. Resource Group

Created a Resource Group to organize all Azure resources used in the assignment.

A Resource Group acts as a logical "folder" where related Azure services are grouped together. In this project, it contains the Event Hubs resources, the Storage Account, and later the Stream Analytics jobs. This makes the setup easier to manage, monitor and clean up.

2. Event Hubs Namespace

Created an Event Hubs Namespace to group and manage the Event Hub used in this project. The namespace acts as the parent "container" for the Event Hub that receives the real-time ATM transaction events generated by the Python script.

Microsoft Azure

t8220066@aueb.gr
AUEB.GR (AUEBGR.ONMICROSO...

Home > Event Hubs

Create Namespace

Event Hubs

Project Details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

Azure for Students

Resource group *

rg-atm-stream-dionisia

Create new

Instance Details

Enter required settings for this namespace, including a price tier and configuring the number of units (capacity).

Namespace name *

ehns-atm-stream-dionisia

.servicebus.windows.net

Region *

France Central

The region selected supports Availability zones. Your namespace will have Availability Zones enabled. [Learn more.](#)

Pricing tier *

Standard

Browse the available plans and their features

Throughput Units *

1

Review + create

< Previous

Next: Advanced >

****Note:** Throughout the setup, I use France Central as the selected region, because it is included in the eligible list of locations available for my Azure for Students subscription, as shown below:

Microsoft Azure

t8220066@aueb.gr
AUEB.GR (AUEBGR.ONMICROSO...

Home > Policy | Assignments

Allowed resource deployment regions

×

Policy Assignment

--

Assignment ID

/subscriptions/9e682a22-95a4-4f1f-b1b9-89291758...

Assigned by

--

Policy

Policy enforcement

Default

Parameters (1)

Resource selectors (0)

Overrides (0)

Exemptions (0)

Remediation (0)

Deployed resou

Search by parameter name

All types

Edit columns

Parameter ID ↑	Parameter n... ↑↓	Parameter value ↑↓	Policy a
listOfAllowedLo...	Allowed locatio...	["uaenorth","austriaeast","polandcentral","francecentral","spaincen...	User de

Give feedback

4 / 32

Home > Event Hubs



Create Namespace

 ...

Event Hubs

✔ Validation succeeded.

- Basics
- Advanced
- Networking
- Tags
- Review + create

Event Hubs Namespace
by Microsoft

Basics

Namespace name	ehns-atm-stream-dionisia
Subscription	Azure for Students
Resource group	rg-atm-stream-dionisia
Region	France Central
Pricing tier	Standard
Throughput Units	1
Availability Zones (Zone Redundancy)	Enabled
Auto-Inflate Maximum Throughput Units	Disabled

Networking

Connectivity method	Public access
---------------------	---------------

Create

< Previous

Next >

ehns-atm-stream-dionisia | Overview

Deployment

Search

Delete Cancel Redeploy Download Refresh

Overview

Inputs

Outputs

Template

✓ Your deployment is complete

Deployment name : ehns-atm-stream-dionisia

Subscription : Azure for Students

Resource group : rg-atm-stream-dionisia

Start time : 6/12/2026, 10:25:36 PM

Correlation ID : 73facdc4-8ca8-4832-a871-82cda27d6bd9

> Deployment details

✓ Next steps

Go to resource

3. Event Hub

Created an Event Hub for real-time ingestion of ATM transaction events.

Home > ehns-atm-stream-dionisia | Event Hubs

Create Event Hub

Event Hubs

Basics Capture Review + create

Event Hub Details

Enter required settings for this event hub, including partition count and message retention.

Name *

eh-atm-transactions ✓

Partition count ⓘ

2

Retention

Configure retention settings for this Event Hub. [Learn more](#)

Cleanup policy ⓘ

Delete ▼

Retention time (hrs) * ⓘ

1

min. 1 hour, max. 168 hours (7days)

Review + create < Previous Next: Capture >

****Note:** The Event Hub was configured with Partition Count = 2.

Partitions define how incoming events are internally distributed and read within the Event Hub. They allow the event stream to be split into multiple parallel sequences.

For example:

- 1 partition → all events are written into one stream
- 2 partitions → events can be distributed across two streams

In the provided *atm_generator.py script*, each event is sent using *accountId* as the *partition key*. This means that all events belonging to the same account are consistently routed to the same partition, while events from different accounts can be distributed across the available partitions.

This configuration is useful because it keeps events for the same account logically grouped, while still allowing the Event Hub to distribute traffic across more than one partition.

Home > ehns-atm-stream-dionisia | Event Hubs

Create Event Hub ...

Event Hubs

✔ Validation succeeded.

Basics

Capture

Review + create

Event Hubs Instance
by Microsoft

Basics

Name	eh-atm-transactions
Partition count	2

Retention

Cleanup policy	Delete
Retention time (hrs)	1

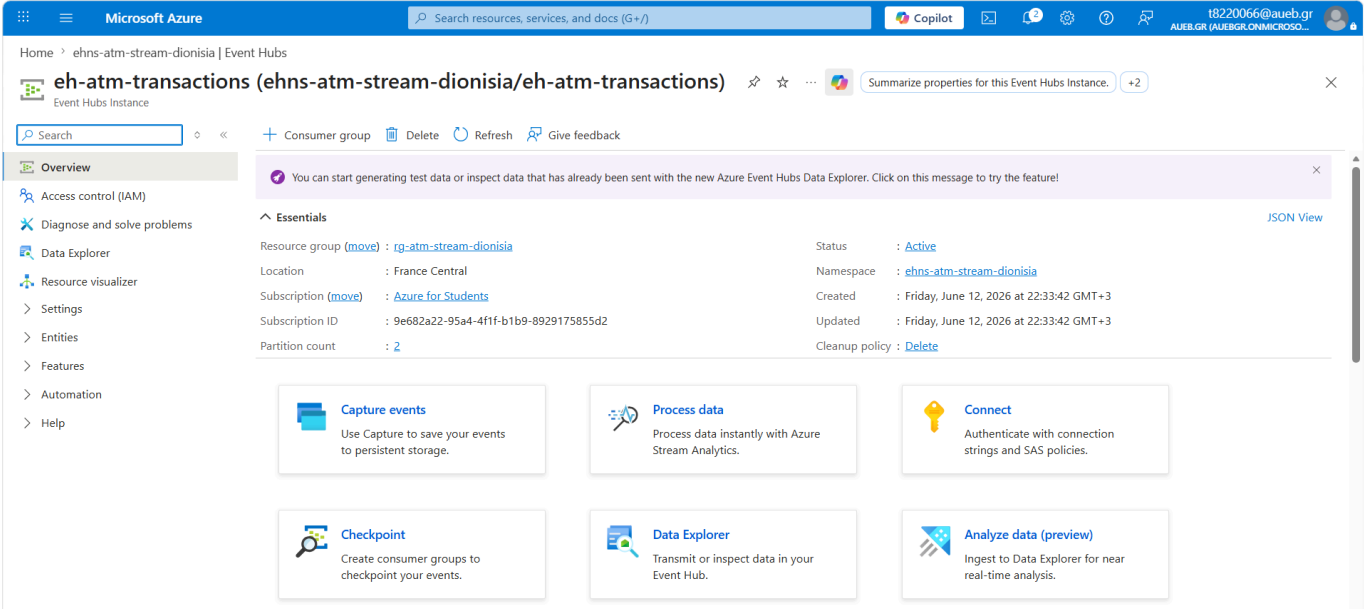
Capture

Capture Status	Off
----------------	-----

Create

< Previous

Next >



4. Shared Access Policy

Created a dedicated send policy for the Python event generator.

The access key is stored locally in `.env` and is not committed to GitHub.

Home

ehns-atm-stream-dionisia | Event Hubs

eh-atm-transactions (ehns-atm-stream-dionisia/eh-atm-transactions)

eh-atm-transactions (ehns-atm-stream-dionisia/eh-atm-transactions)

Event Hubs Instance

Search

◊ ◀

+ Add

Overview

Access control (IAM)

Diagnose and solve problems

Data Explorer

Resource visualizer

Settings

Shared access policies

Configuration

Properties

Locks

Entities

Features

Automation

Help

Search to filter items...

Policy

Claims

no policies have been set up yet.

Add SAS Policy

Event Hubs

Policy name *

send-policy ✓

☐ Manage

☒ Send

☐ Listen

Create

SAS Policy: send-policy



Event Hub



Regenerate primary key



Regenerate secondary key



Delete

Event Hubs access control with Shared Access Signatures [Learn more](#)

Claims



Manage



Send



Listen

Primary key

.....



Secondary key

.....



Primary connection string

.....



Secondary connection string

.....



SAS policy ARM ID

/subscriptions/9e682a22-95a4-4f1f-b1b9-8929175855d2/resourceGroups/rg-atm...

Save

Discard changes



Give feedback

5. Consumer Groups

Created separate consumer groups for the five Stream Analytics jobs, so each job can independently read the same event stream.

- cg-task1
- cg-task2
- cg-task3
- cg-task4
- cg-task5

A Consumer Group can be understood as an independent view of the same Event Hub stream. It does not duplicate the data. Instead, it allows each consumer to keep its own reading position.

For example:

- cg-task1 reads the stream to calculate transaction volume per area
- cg-task2 reads the same stream to detect high-value withdrawals
- cg-task3 reads the same stream to detect burst activity per account

All Stream Analytics jobs read the same ATM events, but each job applies a different query and writes to a different output. By assigning a separate consumer group to each job, every job can process the full stream independently.

If all jobs used the same consumer group, they would share the same reading group, which could cause conflicts or unexpected behavior when multiple readers access the same partitions. For this reason, one consumer group per Stream Analytics job is a cleaner and safer setup.

Each job will later use the same Event Hub as input, but it will write its results to its own Blob container.

Microsoft Azure portal interface showing the 'eh-atm-transactions' Event Hubs instance. The 'Consumer groups' tab is selected, displaying a table with columns 'Name' and 'Location'. The table lists six consumer groups: \$Default, cg-task1, cg-task2, cg-task3, cg-task4, and cg-task5, all located in 'francecentral'. A 'Create consumer group' dialog is open on the right, showing the 'Name' field with 'cg-task1' entered. The 'Create' button is visible at the bottom of the dialog.

Name	Location
\$Default	francecentral
cg-task1	francecentral
cg-task2	francecentral
cg-task3	francecentral
cg-task4	francecentral
cg-task5	francecentral

6. Storage Account

Created a Storage Account to store the JSON outputs produced by the Stream Analytics jobs.

Microsoft Azure

Search resources, services, and docs (G+/)

Home

Create a storage account ...

Subscription *

Azure for Students

Resource group *

rg-atm-stream-dionisia

[Create new](#)

Instance details

Storage account name * ⓘ

statmstreamdionisia4

Region * ⓘ

(Europe) France Central

[Deploy to an Azure Extended Zone](#)

Preferred storage type

Azure Blob Storage or Azure Data Lake Storage

This helps us provide relevant guidance. It doesn't restrict your storage to this resource type. [Learn more](#)

Performance * ⓘ

☒ **Standard:** Recommended for most scenarios (general-purpose v2 account)

☐ **Premium:** Recommended for scenarios that require low latency.

Redundancy * ⓘ

Locally redundant storage (LRS)

Previous

Next

Review + create

Microsoft Azure

Search resources, services, and docs (G+I)

Home

Create a storage account ...

Basics

Advanced

Networking

Data protection

Security

Encryption

Tags

Review + create

View automation template

Basics

Subscription

Resource group

Location

Storage account name

Primary service

Performance

Replication

Azure for Students

rg-atm-stream-dionisia

France Central

statmstreamdionisia4

Azure Blob Storage or Azure Data Lake Storage

Standard

Locally redundant storage (LRS)

Advanced

Enable hierarchical namespace

Enable SFTP

Enable network file system v3

Allow cross-tenant replication

Disabled

Disabled

Disabled

Disabled

Previous

Next

Create

Microsoft Azure

Search resources, services, and docs (G+I)

Copilot

18220066@aubeb.gr

Home > statmstreamdionisia4_1781448159574 | Overview

statmstreamdionisia4

Storage account

Search

Upload

Open in Explorer

Delete

Move

Refresh

Open in mobile

CLI / PS

Feedback

Overview

Activity log

Tags

Diagnose and solve problems

Access Control (IAM)

Data migration

Events

Storage browser

Storage Mover

Partner solutions

Resource visualizer

Data storage

Security + networking

Data management

Settings

Monitoring

Essentials

Resource group (move)

Location

Subscription (move)

Subscription ID

Disk state

Tags (edit)

rg-atm-stream-dionisia

francecentral

Azure for Students

9e682a22-95a4-4f1f-b1b9-8929175855d2

Available

Add tags

Properties

Monitoring

Capabilities (7)

Recommendations (0)

Tutorials

Tools + SDKs

Blob service

Hierarchical namespace

Default access tier

Blob anonymous access

Blob soft delete

Container soft delete

Versioning

Change feed

Disabled

Hot

Disabled

Enabled (7 days)

Enabled (7 days)

Disabled

Disabled

Security

Require secure transfer for REST API operations

Storage account key access

Minimum TLS version

Infrastructure encryption

Enabled

Enabled

Version 1.2

Disabled

Networking

Public network access

Enabled

Deployment succeeded

Deployment 'statmstreamdionisia4_1781448159574' to resource group 'rg-atm-stream-dionisia' was successful.

Go to resource

Go to resource group

JSON View

7. Blob Containers

Created five private Blob containers, one for each Stream Analytics task.

- task1-volume-area

- task2-high-value-withdrawals
- task3-burst-account
- task4-avg-withdrawal-area
- task5-type-distribution

The top screenshot shows the 'New container' dialog in the Azure portal. The 'Name' field is set to 'task1-volume-area' and the 'Anonymous access level' is set to 'Private (no anonymous access)'. A message states: 'The access level is set to private because anonymous access is disabled on this storage account.'

The bottom screenshot shows the 'Containers' page for the 'statmstreamdionisia4' storage account. It displays a table with 6 items:

Name	Last modified	Anonymous access level	Lease state	Storage connector
\$logs	6/14/2026, 5:44:37 PM	Private	Available	-
task1-volume-area	6/14/2026, 5:59:42 PM	Private	Available	-
task2-high-value-withdrawals	6/14/2026, 5:59:57 PM	Private	Available	-
task3-burst-account	6/14/2026, 6:00:08 PM	Private	Available	-
task4-avg-withdrawal-area	6/14/2026, 6:00:19 PM	Private	Available	-
task5-type-distribution	6/14/2026, 6:00:27 PM	Private	Available	-

Event Generator

The event generator is implemented in Python and is located under the directory: `src/atm_generator.py`

This script generates simulated ATM transaction events and sends them continuously to the Azure Event Hub. Each event contains fields such as `eventTime`, `area`, `atmId`, `accountId`, `transactionType`, `amount` and `currency`.

The original configuration values at the beginning of the script were replaced with environment variables. This was done to avoid hardcoding Azure credentials directly in the source code. The script now reads the required values from a local `.env` file:

```
HOST_NAME=...
```

```
SHARED_ACCESS_KEY_NAME=...  
SHARED_ACCESS_KEY=...  
EVENT_HUB_NAME=...
```

The `.env` file is used only locally and is excluded from GitHub through `.gitignore`.

To support loading environment variables from the `.env` file, the `python-dotenv` package was added to `requirements.txt`.

Run

- From the root directory of the project, create and activate a virtual environment:

```
python -m venv .venv  
.venv\Scripts\activate
```

- Install the required dependencies:

```
pip install -r requirements.txt
```

- Run the event generator:

```
python src/atm_generator.py
```

If the configuration is correct, the script connects to the Event Hub and starts sending ATM transaction events continuously, as shown below:

```

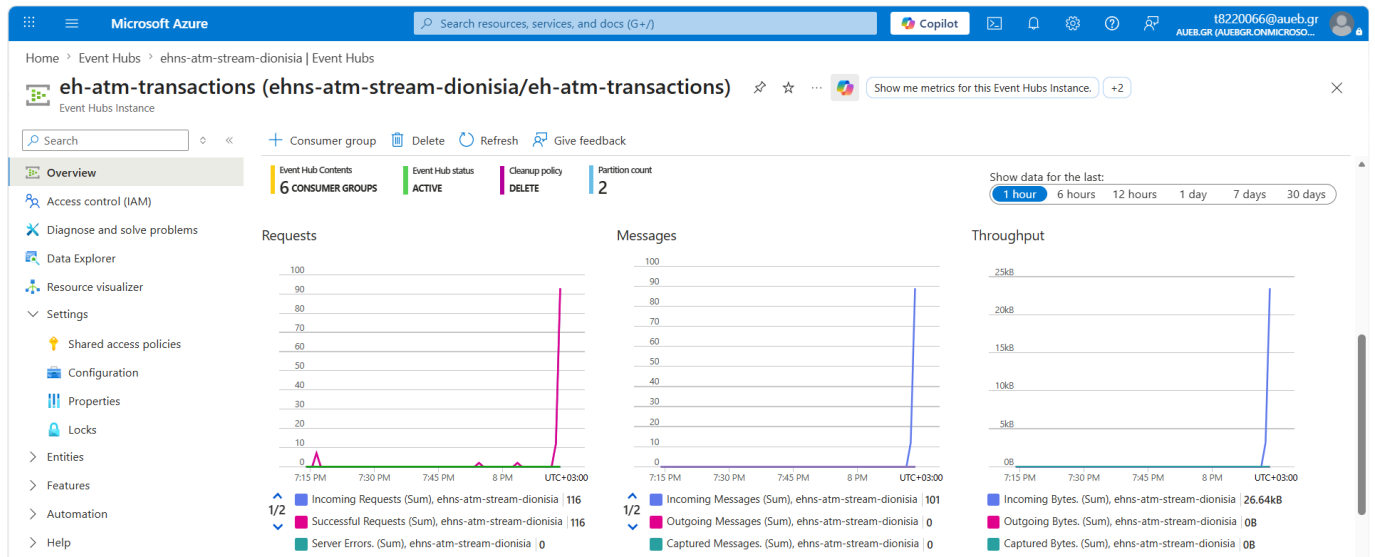
25 }
26
27 missing = [name for name, value in required_vars.items() if not value]
28
29 if missing:
30     raise ValueError(f"Missing required environment variables: ({', '.join(missing)})")
31 # =====
32
33 # Create a standard connection string from the components for the SDK
34 CONNECTION_STR = (
35     f"Endpoint=sb://{HOST_NAME}/;"
36     f"SharedAccessKeyName={SHARED_ACCESS_KEY_NAME};"
37     f"SharedAccessKey={SHARED_ACCESS_KEY}"
38 )
39
40
41 # --- Generator settings ---

```

```

(.venv) PS C:\Users\koron\Documents\BD\Big-Data-Management-Systems\Azure-Stream-Analytics> python src/atm_generator.py
Sent: {"eventTime": "2026-06-14T17:11:11.541328+00:00", "area": "Heraklion", "atmId": "ATM-012", "accountId": "ACC-00435", "transactionType": "deposit", "amount": 800.0, "currency": "EUR"} | Partition Key: ACC-00435
Sent: {"eventTime": "2026-06-14T17:11:12.111866+00:00", "area": "Thessaloniki", "atmId": "ATM-001", "accountId": "ACC-00052", "transactionType": "withdrawal", "amount": 50.0, "currency": "EUR"} | Partition Key: ACC-00052
Sent: {"eventTime": "2026-06-14T17:11:13.046764+00:00", "area": "Athens-North", "atmId": "ATM-003", "accountId": "ACC-00051", "transactionType": "transfer", "amount": 300.0, "currency": "EUR"} | Partition Key: ACC-00051
Sent: {"eventTime": "2026-06-14T17:11:13.666605+00:00", "area": "Heraklion", "atmId": "ATM-005", "accountId": "ACC-00425", "transactionType": "withdrawal", "amount": 150.0, "currency": "EUR"} | Partition Key: ACC-00425
Sent: {"eventTime": "2026-06-14T17:11:14.299996+00:00", "area": "Patras", "atmId": "ATM-010", "accountId": "ACC-00261", "transactionType": "withdrawal", "amount": 50.0, "currency": "EUR"} | Partition Key: ACC-00261
Sent: {"eventTime": "2026-06-14T17:11:14.863822+00:00", "area": "Thessaloniki", "atmId": "ATM-010", "accountId": "ACC-00407", "transactionType": "withdrawal", "amount": 150.0, "currency": "EUR"} | Partition Key: ACC-00407
Sent: {"eventTime": "2026-06-14T17:11:15.478199+00:00", "area": "Athens-North", "atmId": "ATM-005", "accountId": "ACC-00008", "transactionType": "withdrawal", "amount": 40.0, "currency": "EUR"} | Partition Key: ACC-00008
Sent: {"eventTime": "2026-06-14T17:11:16.161422+00:00", "area": "Athens-Center", "atmId": "ATM-008", "accountId": "ACC-00357", "transactionType": "withdrawal", "amount": 80.0, "currency": "EUR"} | Partition Key: ACC-00357

```



Queries

The Stream Analytics queries are stored in the [queries/](#) directory. Each query corresponds to one of the five required tasks and will later be used inside a separate Azure Stream Analytics job.

Basic Structure

All queries follow the same general structure:

```

SELECT
    ...,
System.Timestamp() AS windowEnd
INTO
outputAlias
FROM
inputAlias TIMESTAMP BY eventTime

```

```
WHERE
...
GROUP BY
...,
WindowFunction(...)
```

The **SELECT** clause defines the fields and aggregations that will be written to the output.

The **INTO** clause defines the output alias of the Stream Analytics job. This alias must match the name of the output configured in Azure Stream Analytics.

The **FROM** clause defines the input alias. In this project, the input alias used in the queries is: **atmInput**. This alias will later be configured to read data from the Azure Event Hub.

Each ATM event generated by the Python script contains its own event timestamp in the `eventTime` field. By using **TIMESTAMP BY eventTime**, Azure Stream Analytics processes the events based on the actual event time instead of only using the time when the event arrived in Azure.

The **WHERE** clause is used only when a query needs to filter specific events before aggregation. For example, some tasks only consider withdrawal transactions.

The **GROUP BY** clause defines how the events are grouped before calculating the aggregations. For example, results can be grouped by area, accountId, or transactionType.

The **WindowFunction(...)** defines the time window over which the stream is analyzed. Since streaming data is continuous, the query needs a time window to decide which events belong together for each calculation.

Finally, this field is included in the output:

System.Timestamp() AS windowEnd returns the timestamp associated with the produced result. In windowed aggregations, this is commonly used to indicate the end time of the window for which the result was calculated. This makes the output easier to interpret, because each result row shows not only the aggregation values but also the time period they refer to.

In this project, the input alias is kept the same for all jobs:

atmInput

The output aliases are different for each task:

```
task10output
task20output
task30output
task40output
task50output
```

When the Stream Analytics jobs are created in Azure, the input and output aliases must match exactly the names used inside the SQL queries.

Window Functions

Window functions are necessary in stream processing because the data does not have a fixed end. Events arrive continuously, so calculations must be performed over specific time intervals.

This assignment uses three types of windows: Tumbling, Hopping, and Sliding.

Tumbling Window

A Tumbling Window splits the stream into fixed-size, non-overlapping time intervals. Each event belongs to exactly one window.

Example:

12:00 - 12:01 12:01 - 12:02 12:02 - 12:03

If a query uses:

```
TumblingWindow(minute, 1)
```

then Azure Stream Analytics calculates one result for each complete one-minute interval.

This type of window is useful when the goal is to produce regular summaries, such as the total number of transactions per area every 1 minute.

Hopping Window

A Hopping Window also has a fixed duration, but the windows can overlap. It uses two values: the window size and the hop size.

Example:

```
HoppingWindow(minute, 10, 1)
```

This means:

Window size: 10 minutes Hop size: 1 minute

So, every 1 minute, the query calculates results using the events from the last 10 minutes.

Example intervals:

12:00 - 12:10 12:01 - 12:11 12:02 - 12:12

This type of window is useful when the goal is to continuously monitor recent activity, such as counting high-value withdrawals in each area during the last 10 minutes.

Sliding Window

A Sliding Window looks for events that occur within a moving time interval. It produces results when events enter or leave the window and the aggregation condition is satisfied.

Example:

`SlidingWindow(second, 30)`

This means that Azure Stream Analytics evaluates activity within a 30-second time range.

This type of window is useful for detecting patterns that can happen at any moment, not only at fixed time boundaries. In this project, it is used to detect burst activity, where an account performs 3 or more transactions within 30 seconds.

For example, if the same account performs transactions at:

12:00:05 12:00:15 12:00:25

then all three transactions fall within a 30-second sliding window, and the account can be flagged as suspicious.

Stream Analytics Jobs

Example: Task 1 Stream Analytics Job

For each task, a separate Azure Stream Analytics job was created. Each job follows the same pipeline:

Event Hub input → Stream Analytics query → Blob Storage output

Task 1 is presented as the detailed example, while the same process was repeated for Tasks 2–5 with the corresponding query, consumer group, output alias, and Blob container.

First, the Stream Analytics job for Task 1 was created.

Microsoft Azure

Home > Stream Analytics jobs

New Stream Analytics job ...

Basics

Storage

Tags

Managed Identity

Review + create

Subscription * ⓘ

Azure for Students

Resource group * ⓘ

rg-atm-stream-dionisia

[Create new](#)

Instance details

Name *

asa-task1-volume-area

Region * ⓘ

(Europe) France Central

Hosting environment *

☒ Cloud

☐ Edge

Streaming unit details

Streaming units (SUs) represents the computing resources that are allocated to execute a Stream Analytics job. The higher the number of SUs, the more CPU and memory resources are allocated for your job. The number of SUs can be modified once you create the job. You will be charged for the job's Streaming Units only when the job runs. [Learn more about streaming units](#) ⓘ

All new Stream Analytics jobs created through the portal use Standard V2 pricing. [Visit Stream Analytics's pricing page to learn more.](#) ⓘ

[For more options, visit here](#) ⓘ

Streaming units *

1

Previous

Next

Review + create

Microsoft Azure

Search

Home > Stream Analytics jobs

New Stream Analytics job

...

Basics

Storage

Tags

Managed Identity

Review + create

View automation template

Basics

Subscription

Azure for Students

Resource group

rg-atm-stream-dionisia

Name

asa-task1-volume-area

Location

France Central

Hosting environment

Cloud

Streaming units

1

Previous

Next

Create

Home

StreamAnalyticsJob | Overview

Deployment

Search

Delete Cancel Redeploy Download Refresh

Overview

Inputs

Outputs

Template

✓ Your deployment is complete

Deployment name : StreamAnalyticsJob
Subscription : Azure for Students
Resource group : rg-atm-stream-dionisia

Start time : 6/14/2026, 8:16:03 PM
Correlation ID : d60d9810-7e32-45b9-b719-c76b1f04c31a

> Deployment details

✓ Next steps

[Go to resource](#)

Input Configuration

The input was configured to read events from the Azure Event Hub. The input alias was set to `atmInput`, which matches the alias used in the SQL query.

For Task 1, the consumer group `cg-task1` was selected, so this job can independently read the same event stream without interfering with the other Stream Analytics jobs.

Home > StreamAnalyticsJob | Overview > asa-task1-volume-area

asa-task1-volume-area | Inputs

Stream Analytics job

Search

+ Add input Refresh

Alias ↑	Source type	Type
---------	-------------	------

Event Hub

New input

Input alias *

atmInput ✓

☐ Provide Event Hub settings manually

☒ Select Event Hub from your subscriptions

Subscription

Azure for Students

Event Hub namespace *

ehns-atm-stream-dionisia

Event Hub name *

☐ Create new ☒ Use existing

eh-atm-transactions

Event Hub consumer group *

☐ Create new ☒ Use existing

cg-task1

Authentication mode

Connection string

[Save](#)

Event Hub



New input

Input alias *

atmInput



- ☐ Provide Event Hub settings manually
- ☒ Select Event Hub from your subscriptions

Subscription

Azure for Students



Event Hub namespace * ⓘ

ehns-atm-stream-dionisia



Event Hub name * ⓘ

- ☐ Create new ☒ Use existing

eh-atm-transactions



Event Hub consumer group * ⓘ

- ☐ Create new ☒ Use existing

cg-task1



Authentication mode

Connection string



Save

Event Hub

×

New input

Event Hub policy name *

ⓘ

☒ Create new

☐ Use existing

asa-task1-volume-area_atmInput_policy

Event Hub policy key ⓘ

.....

Partition key ⓘ

Event serialization format *

ⓘ

JSON

▼

Encoding ⓘ

UTF-8

▼

Event compression type ⓘ

None

▼

Schema registry

None

▼

Save

asa-task1-volume-area | Inputs ☆ ...

Stream Analytics job

Search

◊ ◀ + Add input ◻ ⌛ Refresh

Overview

Activity log

Access control (IAM)

Alias 1	Source type	Type	A...	R...
atmInput	Stream	Event Hub	Conr	

Output Configuration

The output was configured to write the query results to Azure Blob Storage. The output alias was set to task1Output, which matches the INTO task1Output clause in the query.

The selected Blob container for this task was task1-volume-area.

The screenshot displays the Microsoft Azure portal interface. The top navigation bar includes the Microsoft Azure logo, a search bar, and a Copilot button. The user's profile and email address (t8220066@aub.gr) are visible in the top right corner.

The main content area is titled "asa-task1-volume-area | Outputs" and shows a table with columns "Alias" and "Type". The "Outputs" section in the left-hand navigation pane is highlighted.

A "Blob storage/ADLS Gen2" configuration panel is open on the right side of the screen. It contains the following fields and options:

- Output alias ***: task1Output (with a green checkmark)
- Provide Blob storage/ADLS Gen2 settings manually**: ☐
- Select Blob storage/ADLS Gen2 from your subscriptions**: ☒
- Subscription**: Azure for Students (dropdown menu)
- Storage account ***: statmstreamdionisia4 (dropdown menu)
- Container ***: ☐ Create new, ☒ Use existing
- Container ***: task1-volume-area (dropdown menu)
- Authentication mode**: Connection string (dropdown menu)
- Storage account key**: (password field with masked characters)
- Event serialization format ***: (dropdown menu)
- Save**: (blue button)

At the bottom of the page, there is a footer that reads "Add or remove favorites by pressing Ctrl+Shift+F".

Blob storage/ADLS Gen2



New output

Output alias *

task1Output



Provide Blob storage/ADLS Gen2 settings manually



Select Blob storage/ADLS Gen2 from your subscriptions

Subscription

Azure for Students



Storage account *

statmstreamdionisia4



Container * ⓘ



Create new



Use existing

task1-volume-area



Authentication mode

Connection string



Storage account key ⓘ

.....

Event serialization format * ⓘ

Save

Blob storage/ADLS Gen2



New output

Event serialization format * ⓘ

JSON



Format ⓘ

Line separated



Encoding ⓘ

UTF-8



Write mode * ⓘ



Once, when all results for the time partition are available.
Ensures exactly once delivery (preview)



Append, as results arrive

Path pattern ⓘ

task1-output



Date format

YYYY/MM/DD



Time format

HH



Minimum rows ⓘ

Save

Query Configuration

The Task 1 query was added to the Stream Analytics job. This query calculates the number of transactions and the total transaction amount per area using a one-minute tumbling window.

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

t8220066@aubeb.gr

AUEB.GR (AUEBGR.ONMICROSO...

Home > StreamAnalyticsJob | Overview > asa-task1-volume-area

asa-task1-volume-area | Query

Stream Analytics job

Search

Start job Open in VS Code Diagnostic settings Refresh Query language docs Share feedback Tutorial

Job ready to start

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Resource visualizer

Job topology

Inputs

Functions

Query

Outputs

No-code editor (preview)

Settings

Developer tools

Monitoring

Inputs (1)

atmInput

Outputs (1)

task1Output

Functions (0)

Test query Save query Discard changes

```
1 SELECT
2   area,
3   COUNT(*) AS transactionCount,
4   SUM(amount) AS totalAmount,
5   System.Timestamp() AS windowEnd
6 INTO
7   task1Output
8 FROM
9   atmInput TIMESTAMP BY eventTime
10 GROUP BY
11   area,
12   TumblingWindow(minute, 1)
```

Input preview Test results Job simulation (preview)

Showing sample events from 'atmInput'.

Table Raw Refresh Select time range Upload sample input Send events

eventTime	area	atmid	accountid	transactionType	amount
datetime	string	string	string	string	bigint
2026-06-14T17:30:58...	Athens-North	ATM-015	ACC-00026	withdrawal	40
2026-06-14T17:31:00...	Athens-North	ATM-011	ACC-00393	withdrawal	300

Job Execution

After configuring the input, output, and query, the job overview was checked to confirm that the configuration was complete.

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

t8220066@aubeb.gr

AUEB.GR (AUEBGR.ONMICROSO...

Home

Stream Analytics jobs

Filter resources by region Browse all resources in this subscription Are any resources impacted by outages

Create Manage view Refresh Export to CSV Open query Assign tags Add to service group

Filter for any field... Subscription equals all Resource Group equals all Location equals all Add filter

Name	Resource Group	Location	Status	Type	Compatibility level	Created (UTC)	Output watermark (...)
asa-task1-volume-area	rg-atm-stream-dionisia	France Central	Running	Cloud	1.2	2026-06-14 17:17:14	

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

t8220066@aubeb.gr

AUEB.GR (AUEBGR.ONMICROSO...

Home > StreamAnalyticsJob | Overview

asa-task1-volume-area

Stream Analytics job

Search

Start job Delete Move Refresh Share feedback

Job ready to start

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Resource visualizer

Job topology

Inputs

Functions

Query

Outputs

No-code editor (preview)

Settings

Developer tools

Monitoring

Created

Essentials

Resource group (move) : rg-atm-stream-dionisia

Location : France Central

Status : Created

Subscription (move) : Azure for Students

Subscription ID : 9e682a22-95a4-4f1f-b1b9-8929175855d2

Pricing plan : StandardV2 (manage)

Tags (edit) : Add tags

Created : Sunday, June 14, 2026 8:17 PM

Started :

Output watermark :

Cluster : Shared

Hosting environment : Cloud

Virtual Network : Disabled

JSON View

Get started Properties Monitoring Tutorials

Inputs (1)

atmInput

Query

Edit query

Outputs (1)

task1Output

Settings

Environment : Standard (multi-tenant)

Storage account settings : -

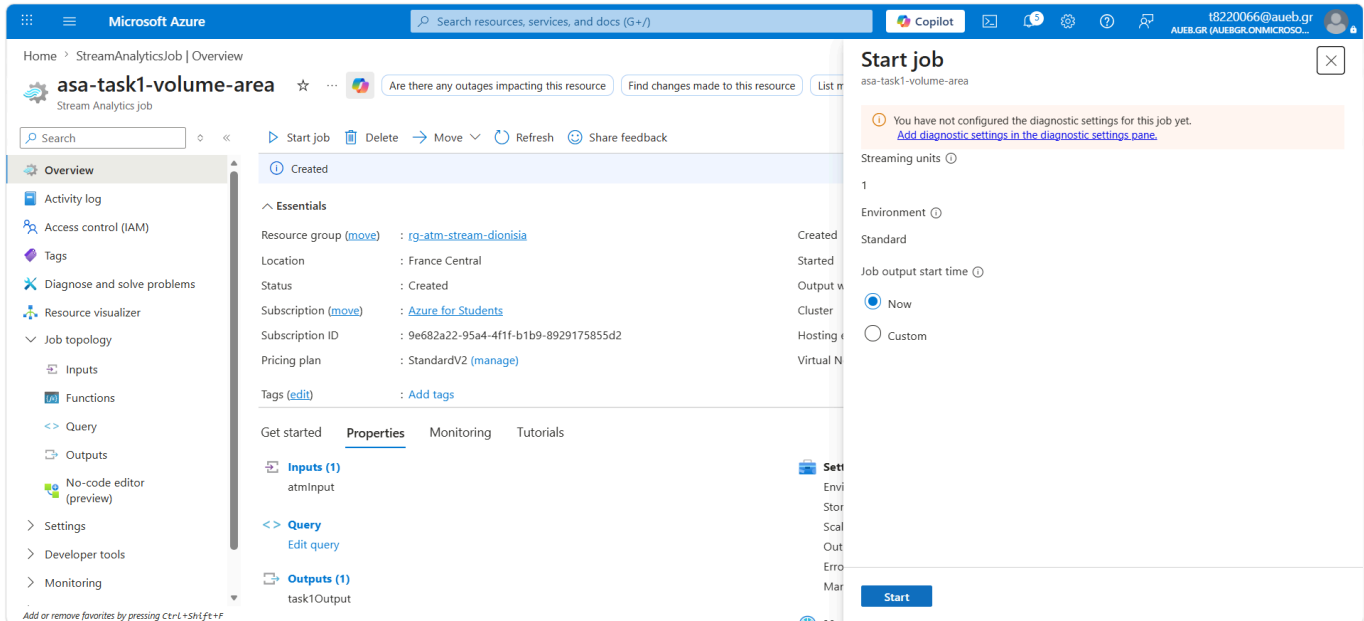
Scale : 1 streaming units

Out of order tolerance : 0 seconds

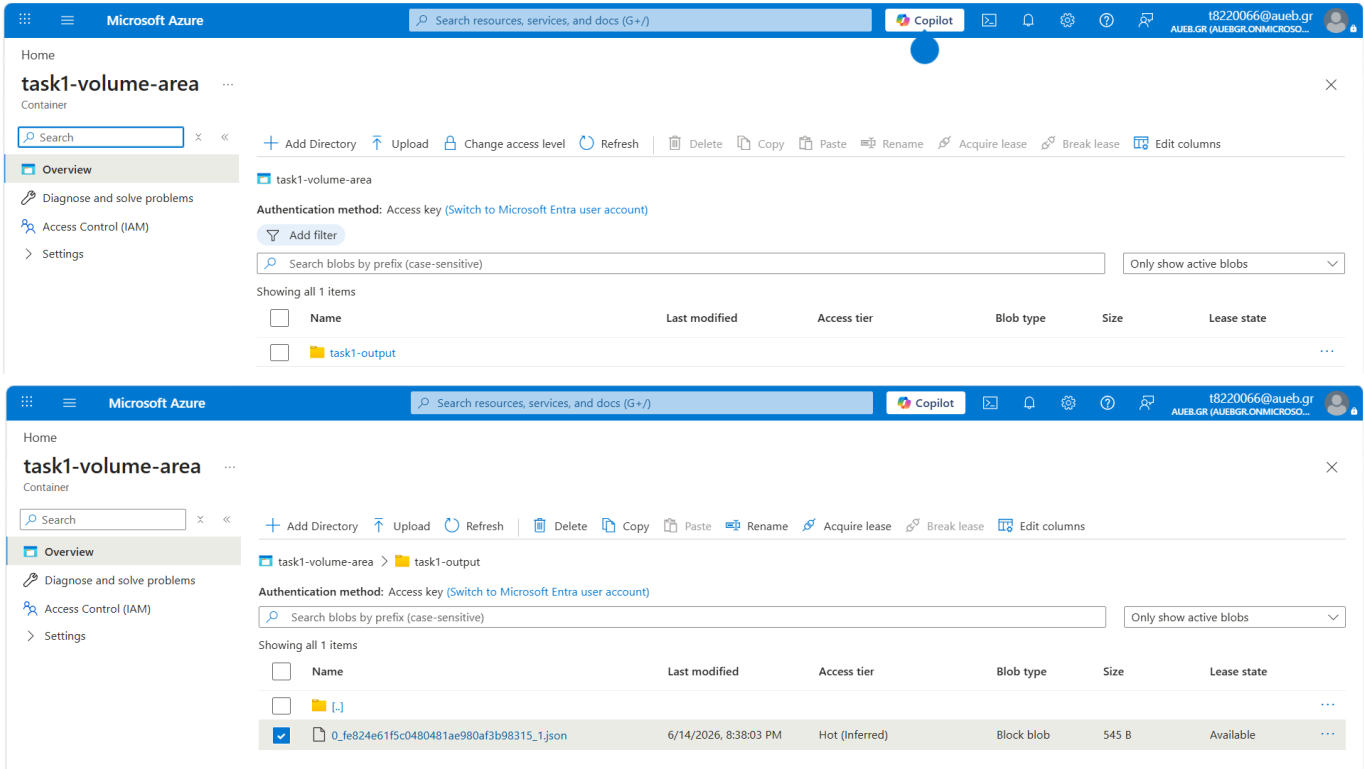
Error policy : Retry

Managed Identity : dbc7774f-8351-470c-a324-6b4948bf5dab

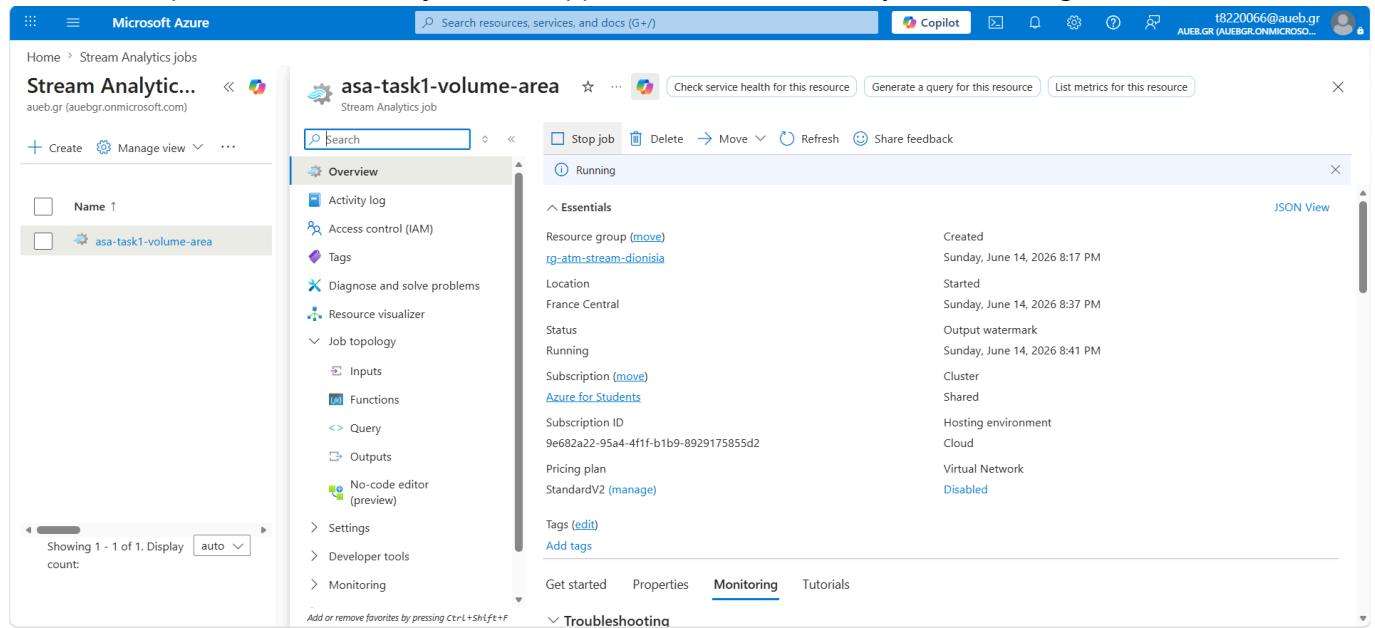
The job was then started using the current time as the start time.



After a few minutes, the job produced output files in the corresponding Blob Storage container.



Once the output was verified, the job was stopped to avoid unnecessary resource usage.



Outputs

The generated output files from Azure Blob Storage were downloaded and stored locally in the **outputs/** directory. Each task has its own output folder, matching the structure of the Blob containers.

```
outputs/  
├─ task1-volume-area/  
├─ task2-high-value-withdrawals/  
├─ task3-burst-account/  
├─ task4-avg-withdrawal-area/  
└─ task5-type-distribution/
```

Overview of all Tasks

The same process was repeated for the remaining tasks. Each task uses the same Event Hub as input, but a different consumer group, output alias, Blob container, and SQL query.

The screenshots below show the final overview of the Stream Analytics jobs and confirm that the jobs were created successfully.

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

t8220066@aubegr.gr

Home

Stream Analytics jobs

Filter resources by region

Subscription equals all

Resource Group equals all

Location equals all

Add filter

Name	Resource Group	Location	Status	Type	Compatibility level	Created (UTC)	Output watermark (...)
asa-task1-volume-area	rg-atm-stream-dionisia	France Central	Stopped	Cloud	1.2	2026-06-14 17:17:14	2026-06-14 17:42:00
asa-task2-high-value-withdrawals	rg-atm-stream-dionisia	France Central	Stopped	Cloud	1.2	2026-06-14 17:45:47	2026-06-14 17:52:00
asa-task3-burst-account	rg-atm-stream-dionisia	France Central	Stopped	Cloud	1.2	2026-06-14 17:48:35	2026-06-14 17:56:32
asa-task4-avg-withdrawal-area	rg-atm-stream-dionisia	France Central	Running	Cloud	1.2	2026-06-14 17:51:32	
asa-task5-type-distribution	rg-atm-stream-dionisia	France Central	Running	Cloud	1.2	2026-06-14 17:58:25	

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

t8220066@aubegr.gr

Home

Stream Analytics jobs

Filter resources by region

Subscription equals all

Resource Group equals all

Location equals all

Add filter

Name	Resource Group	Location	Status	Type	Compatibility level	Created (UTC)	Output watermark (...)
asa-task1-volume-area	rg-atm-stream-dionisia	France Central	Stopped	Cloud	1.2	2026-06-14 17:17:14	2026-06-14 17:42:00
asa-task2-high-value-withdrawals	rg-atm-stream-dionisia	France Central	Stopped	Cloud	1.2	2026-06-14 17:45:47	2026-06-14 17:52:00
asa-task3-burst-account	rg-atm-stream-dionisia	France Central	Stopped	Cloud	1.2	2026-06-14 17:48:35	2026-06-14 17:56:32
asa-task4-avg-withdrawal-area	rg-atm-stream-dionisia	France Central	Stopped	Cloud	1.2	2026-06-14 17:51:32	2026-06-14 18:10:00
asa-task5-type-distribution	rg-atm-stream-dionisia	France Central	Stopped	Cloud	1.2	2026-06-14 17:58:25	2026-06-14 18:10:00